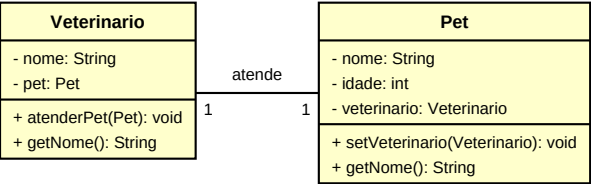


1. Associação 1 : 1 (um para um)

Cada veterinário atende exatamente um pet, e cada pet tem exatamente um veterinário.



Regra: multiplicidade 1 de cada lado -> atributo simples nas duas classes.
Os dois lados guardam a referência um do outro (associação bidirecional).

linhas que mudam de uma associação para a outra

```
Veterinario.java + Pet.java
public class Veterinario {
    private String nome;
    private Pet pet; // 1 única referência

    public Veterinario(String nome) {
        this.nome = nome;
    }

    public void atenderPet(Pet pet) {
        this.pet = pet;
        pet.setVeterinario(this); // mantém os dois lados
    }

    public String getNome() { return nome; }
}

public class Pet {
    private String nome;
    private int idade;
    private Veterinario veterinario; // 1 única referência

    public Pet(String nome, int idade) {
        this.nome = nome;
        this.idade = idade;
    }

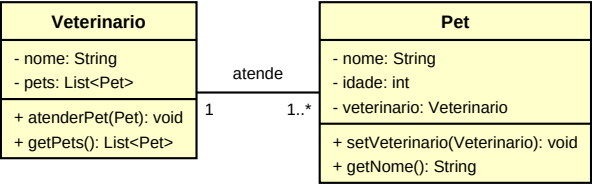
    public void setVeterinario(Veterinario v) {
        this.veterinario = v;
    }

    public String getNome() { return nome; }
}

// Uso
Veterinario vet = new Veterinario("Dra. Ana");
Pet rex = new Pet("Rex", 3);
vet.atenderPet(alex);
```

2. Associação 1 : N (um para muitos)

Um veterinário atende vários pets; cada pet é atendido por um único veterinário.



Regra: o lado com multiplicidade * vira uma coleção (List, Set...).
O lado com multiplicidade 1 continua sendo um atributo simples.

linhas que mudam de uma associação para a outra

```
Veterinario.java + Pet.java
import java.util.ArrayList;
import java.util.List;

public class Veterinario {
    private String nome;
    private List<Pet> pets = new ArrayList<>(); // muitos pets

    public Veterinario(String nome) {
        this.nome = nome;
    }

    public void atenderPet(Pet pet) {
        pets.add(pet);
        pet.setVeterinario(this);
    }

    public List<Pet> getPets() { return pets; }
}

public class Pet {
    private String nome;
    private int idade;
    private Veterinario veterinario; // apenas 1 veterinário

    public Pet(String nome, int idade) {
        this.nome = nome;
        this.idade = idade;
    }

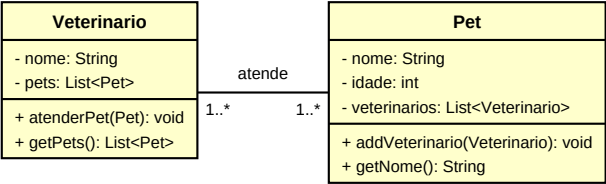
    public void setVeterinario(Veterinario v) {
        this.veterinario = v;
    }

    public String getNome() { return nome; }
}

// Uso
Veterinario vet = new Veterinario("Dra. Ana");
vet.atenderPet(new Pet("Rex", 3));
vet.atenderPet(new Pet("Mel", 5));
```

3. Associação N : N (muitos para muitos)

Um veterinário atende vários pets e um pet pode ser atendido por vários veterinários.



Regra: * dos dois lados -> coleção nas duas classes.
O teste contains() evita duplicatas e a recursão infinita entre os métodos.
No banco de dados, esse caso vira uma tabela associativa (ex.: atendimento).

linhas que mudam de uma associação para a outra

```
Veterinario.java + Pet.java
import java.util.ArrayList;
import java.util.List;

public class Veterinario {
    private String nome;
    private List<Pet> pets = new ArrayList<>(); // muitos pets

    public Veterinario(String nome) { this.nome = nome; }

    public void atenderPet(Pet pet) {
        if (!pets.contains(pet)) {
            pets.add(pet);
            // contains() evita duplicata e laço infinito
            pet.addVeterinario(this);
        }
    }

    public List<Pet> getPets() { return pets; }
}

public class Pet {
    private String nome;
    private int idade;
    private List<Veterinario> veterinarios = new ArrayList<>();

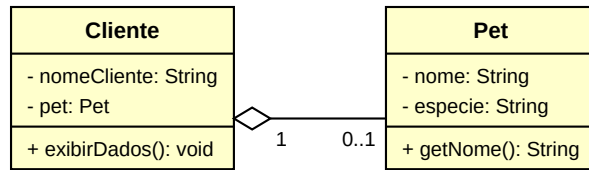
    public Pet(String nome, int idade) {
        this.nome = nome;
        this.idade = idade;
    }

    public void addVeterinario(Veterinario v) {
        if (!veterinarios.contains(v)) {
            veterinarios.add(v);
            v.atenderPet(this);
        }
    }
}

// Uso
Veterinario ana = new Veterinario("Dra. Ana");
Veterinario bru = new Veterinario("Dr. Bruno");
Pet rex = new Pet("Rex", 3);
ana.atenderPet(alex);
bru.atenderPet(alex);
```

4. Agregação (losango vazado)

O Cliente TEM um Pet, mas o Pet tem vida própria: existe antes e depois do Cliente.



Losango VAZADO no lado do todo (Cliente) = agregação.

Marca no código: o objeto chega PRONTO pelo construtor/setter (new fora).

Ciclo de vida independente: destruir o Cliente não destrói o Pet.

linhas que caracterizam a agregação

Cliente.java + Pet.java

```
public class Pet {
    private String nome;
    private String especie;

    public Pet(String nome, String especie) {
        this.nome = nome;
        this.especie = especie;
    }

    public String getNome() { return this.nome; }
}

public class Cliente {
    private String nomeCliente;
    // o Pet existe independentemente do Cliente
    private Pet pet;

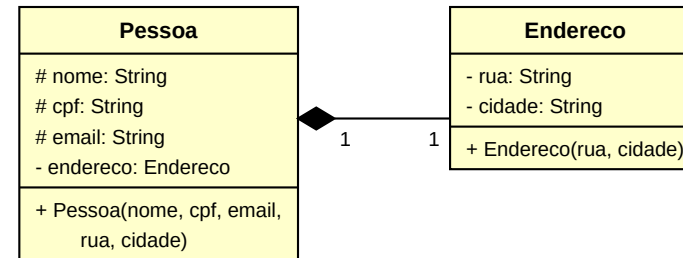
    // recebe um Pet JÁ CRIADO como parâmetro
    public Cliente(String nome, Pet pet) {
        this.nomeCliente = nome;
        this.pet = pet;
    }

    public void exibirDados() {
        System.out.println("Cliente: " + this.nomeCliente
            + " | Pet: " + pet.getNome());
    }
}

// Uso
Pet rex = new Pet("Rex", "Cachorro");
Cliente ana = new Cliente("Ana", rex);
ana.exibirDados();
// se 'ana' for descartada, 'rex' continua existindo
// e pode ser passado para outro Cliente
```

5. Composição (losango preenchido)

A Pessoa TEM um Endereço que não faz sentido sozinho: nasce e morre com ela.



Losango PREENCHIDO no lado do todo (Pessoa) = composição.

Marca no código: o new acontece DENTRO da classe todo; a parte não vem de fora.

Ciclo de vida dependente: sem Pessoa, aquele Endereço deixa de existir.

linhas que caracterizam a composição

Pessoa.java + Endereco.java

```
class Endereco {
    private String rua;
    private String cidade;

    // o Endereco não existe fora da Pessoa (dependência total)
    public Endereco(String rua, String cidade) {
        this.rua = rua;
        this.cidade = cidade;
    }
}

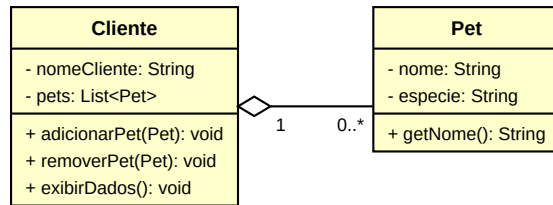
class Pessoa {
    protected String nome;
    protected String cpf;
    protected String email;
    // Composicao: Pessoa "tem um" Endereco
    private Endereco endereco;

    public Pessoa(String nome, String cpf, String email,
        String rua, String cidade) {
        this.nome = nome;
        this.cpf = cpf;
        this.email = email;
        // a Pessoa CRIA o próprio Endereço
        this.endereco = new Endereco(rua, cidade);
    }
}

// Uso
Pessoa p = new Pessoa("Ana", "000.000.000-00",
    "ana@email.com",
    "Rua das Flores, 10", "Curitiba");
// não existe referência externa ao Endereco:
// ele nasce e é destruído junto com a Pessoa
```

6. Agregação 1 : 0..* (losango vazado)

Um Cliente pode ter nenhum, um ou vários Pets; cada Pet continua existindo por conta própria.



Multiplicidade 0..* -> coleção que pode ficar vazia (nenhum Pet).

Continua sendo agregação: o new do Pet acontece fora e o objeto chega pronto.

remove() tira o Pet da lista, mas não destrói o objeto.

linhas que caracterizam a agregação 1 : 0..*

Cliente.java + Pet.java

```
import java.util.ArrayList;
import java.util.List;

public class Pet {
    private String nome;
    private String especie;
    public Pet(String nome, String especie) {
        this.nome = nome;
        this.especie = especie;
    }
    public String getNome() { return this.nome; }
}

public class Cliente {
    private String nomeCliente;
    // 0 ou N pets, todos criados FORA do Cliente
    private List<Pet> pets = new ArrayList<>();

    // pode nascer sem nenhum pet -> mínimo 0
    public Cliente(String nome) {
        this.nomeCliente = nome;
    }

    // recebe um Pet JA CRIADO
    public void adicionarPet(Pet pet) {
        pets.add(pet);
    }

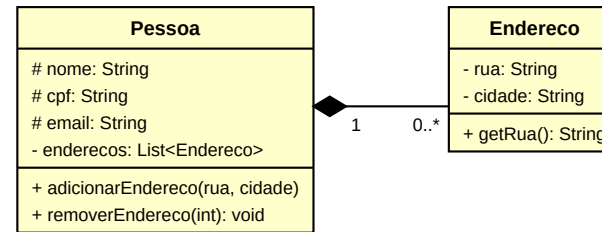
    public void removerPet(Pet pet) {
        pets.remove(pet); // o Pet continua existindo
    }

    public void exibirDados() {
        System.out.print("Cliente: " + this.nomeCliente);
        for (Pet p : pets) System.out.print(" | " + p.getNome());
    }
}

// Uso
Pet rex = new Pet("Rex", "Cachorro");
Cliente ana = new Cliente("Ana"); // 0 pets
ana.adicionarPet(rex); // agora 1
ana.removerPet(rex); // 'rex' sobrevive sozinho
```

7. Composição 1 : 0..* (losango preenchido)

Uma Pessoa pode ter nenhum, um ou vários Endereços, mas todos pertencem só a ela.



Multiplicidade 0..* -> coleção que pode ficar vazia (nenhum Endereço).

Continua sendo composição: o método recebe DADOS e faz o new lá dentro.

Nada de fora guarda referência: remover da lista destrói o Endereço.

linhas que caracterizam a composição 1 : 0..*

Pessoa.java + Endereco.java

```
import java.util.ArrayList;
import java.util.List;

class Endereco {
    private String rua;
    private String cidade;
    // construtor sem public: so a Pessoa consegue criar
    Endereco(String rua, String cidade) {
        this.rua = rua;
        this.cidade = cidade;
    }
    public String getRua() { return this.rua; }
}

class Pessoa {
    protected String nome;
    protected String cpf;
    protected String email;
    // 0 ou N enderecos, todos criados DENTRO da Pessoa
    private List<Endereco> enderecos = new ArrayList<>();

    public Pessoa(String nome, String cpf, String email) {
        this.nome = nome;
        this.cpf = cpf;
        this.email = email;
    }

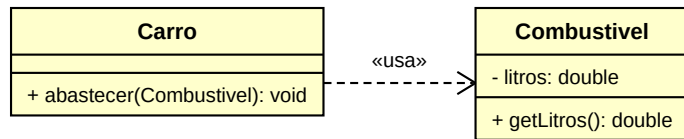
    // recebe DADOS, nao objetos: a propria Pessoa da new
    public void adicionarEndereco(String rua, String cidade) {
        enderecos.add(new Endereco(rua, cidade));
    }

    public void removerEndereco(int i) {
        enderecos.remove(i); // o Endereco deixa de existir
    }
}

// Uso
Pessoa p = new Pessoa("Ana", "000...", "ana@email.com");
p.adicionarEndereco("Rua das Flores, 10", "Curitiba");
p.adicionarEndereco("Av. Brasil, 500", "Londrina");
// se 'p' deixar de existir, os dois Enderecos somem junto
```

8. Dependência - envio de UM objeto

O Carro usa um Combustível recebido como parâmetro, sem guardá-lo como atributo.



Seta TRACEJADA com ponta aberta e estereótipo «usa» = dependência.
Não há vínculo estrutural: só uso momentâneo de 1 objeto por chamada.
O objeto vive apenas enquanto o método executa (relação mais fraca de todas).

linhas que mostram a ausência de vínculo estrutural

parâmetro que cria a dependência

Carro.java + Combustivel.java

```
public class Combustivel {
    private double litros;

    public Combustivel(double litros) {
        this.litros = litros;
    }

    public double getLitros() { return this.litros; }
}

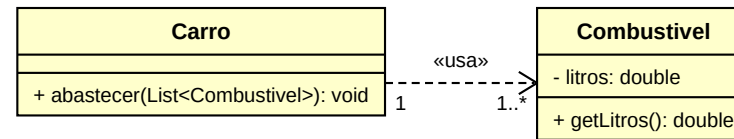
public class Carro {

    // nenhum atributo do tipo Combustivel:
    // a classe apenas USA o objeto durante o metodo
    public void abastecer(Combustivel c) {
        System.out.println(c.getLitros());
    }
}

// Uso
Carro carro = new Carro();
carro.abastecer(new Combustivel(45.0));
// terminado o metodo, 'c' sai de escopo e o Carro
// nao guarda nenhuma referencia ao Combustivel
```

9. Dependência - envio de N objetos

O Carro usa VÁRIOS Combustíveis recebidos numa lista, sem guardá-los como atributo.



Multiplicidade 1 : 1..* nas pontas = UM Carro usa VÁRIOS Combustíveis.
Continua sendo dependência: seta tracejada «usa», a lista chega por parâmetro.
Regra prática: se a lista virasse atributo, seria associação, não dependência.

linhas que mudam em relação ao item 8

parâmetro que cria a dependência

Carro.java + Combustivel.java

```
import java.util.List;

public class Combustivel {
    private double litros;

    public Combustivel(double litros) {
        this.litros = litros;
    }

    public double getLitros() { return this.litros; }
}

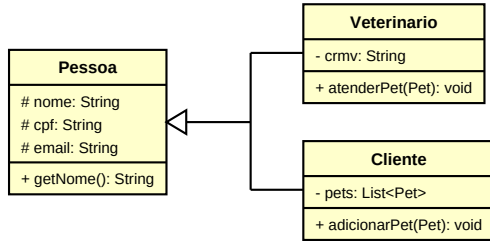
public class Carro {

    // continua sem atributo: muda so a forma do parametro
    public void abastecer(List<Combustivel> tanques) {
        for (Combustivel c : tanques) {
            System.out.println(c.getLitros());
        }
    }

    // Uso
    Carro carro = new Carro();
    carro.abastecer(List.of(new Combustivel(45.0),
                           new Combustivel(12.5)));
    // a lista e os objetos continuam fora do Carro:
    // 1 Carro usa N Combustiveis apenas durante o metodo
```

10. Herança - extends (é-um)

Veterinario e Cliente SÃO Pessoa: herdaram atributos e métodos e acrescentam o que é seu.



Triângulo VAZADO em linha cheia, apontando para a superclasse = generalização.

Uma superclasse pode ter quantas subclasses forem necessárias (aqui, duas).

Sem multiplicidade: não é um objeto dentro de outro, e sim um tipo derivado de outro.

'protected' (#) libera o atributo para as subclasses; super(...) chama o construtor pai.

linhas que caracterizam a herança

palavra-chave que estabelece o vínculo

Pessoa.java + Veterinario.java + Cliente.java

```
import java.util.ArrayList;
import java.util.List;

public class Pessoa { // superclasse
    protected String nome; // visível nas subclasses
    protected String cpf;
    protected String email;

    public Pessoa(String nome, String cpf, String email) {
        this.nome = nome;
        this.cpf = cpf;
        this.email = email;
    }

    public String getNome() { return this.nome; }
}

public class Veterinario extends Pessoa {
    private String crmv; // atributo so da subclasse

    public Veterinario(String nome, String cpf,
        String email, String crmv) {
        super(nome, cpf, email); // reaproveita a superclasse
        this.crmv = crmv;
    }

    public void atenderPet(Pet pet) {
        // 'nome' vem herdado de Pessoa
        System.out.println(nome + " atende " + pet.getNome());
    }
}

public class Cliente extends Pessoa {
    private List<Pet> pets = new ArrayList<>();

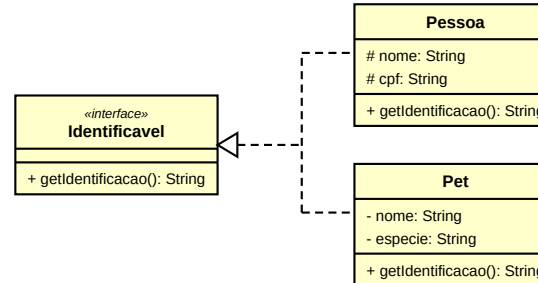
    public Cliente(String nome, String cpf, String email) {
        super(nome, cpf, email); // mesma superclasse
    }

    public void adicionarPet(Pet pet) { pets.add(pet); }
}

// Uso: polimorfismo - os dois SAO Pessoa
Pessoa[] pessoas = { new Veterinario("Ana", "000...",
    "ana@email.com", "PR-1234"),
    new Cliente("Bruno", "111...", "b@email.com") };
for (Pessoa p : pessoas)
    System.out.println(p.getNome()); // metodo herdado
```

11. Interface - implements (sabe-fazer)

Pessoa e Pet não têm parentesco, mas ambas assinam o mesmo contrato Identificavel.



Triângulo VAZADO em linha TRACEJADA apontando para a interface = realização.

A interface só define assinaturas; cada classe escreve sua própria implementação.

Extends: herda comportamento (é-um). Implements: assume um contrato (sabe-fazer).

Uma classe estende UMA superclasse, mas pode implementar VÁRIAS interfaces.

linhas que caracterizam o contrato

palavra-chave que estabelece o vínculo

Identificavel.java + Pessoa.java + Pet.java

```
import java.util.List;

public interface Identificavel {
    // so a assinatura: nenhum codigo, nenhum atributo
    String getIdentificacao();
}

public class Pessoa implements Identificavel {
    protected String nome;
    protected String cpf;

    public Pessoa(String nome, String cpf) {
        this.nome = nome;
        this.cpf = cpf;
    }

    @Override
    public String getIdentificacao() {
        return "CPF " + this.cpf;
    }
}

public class Pet implements Identificavel {
    private String nome;
    private String especie;

    public Pet(String nome, String especie) {
        this.nome = nome;
        this.especie = especie;
    }

    @Override
    public String getIdentificacao() {
        return this.nome + " (" + this.especie + ")";
    }
}

// Uso: mesmo contrato para tipos sem parentesco entre si
List<Identificavel> lista = List.of(
    new Pessoa("Ana", "000..."), new Pet("Rex", "Cao"));
for (Identificavel i : lista)
    System.out.println(i.getIdentificacao());
```